OPTIMIZE ANYTHING

Assignment 8 - Ten-track experimental portfolio

Main case study: capacity-constrained campus shuttle routing. Development: AI-assisted implementations, independent tests, measured comparisons and documented corrections.

Evidence and interpretation

This portfolio contains executable experiments for all ten compulsory tracks. Exact solutions, numerical references, native program outputs, SQL result hashes and real xv6 test logs provide correctness evidence. Negative results are retained: tiny parallel workloads can be slower, quantum classifiers can lose to classical models, and line-preserving test reduction can lose fault detection.

The second-tool output is supplied as Gemini 3.8 Flash (High). Its original code and metadata claims are preserved. The student confirms that this work was done in Google Antigravity IDE. Original screenshots in the appendix show the selected model, prompt execution and response. Codex independently reran checks and found large-integer errors and unsupported analysis claims. See the AI comparison and prompts/second-tool/ANTIGRAVITY_EVIDENCE.md.

Submission repository: https://github.com/Ninjabeam20/OT_Assign_8. The verified publication status is recorded in docs/SUBMISSION_STATUS.md.

Reproducibility

Python 3.12 with uv.lock; Apple Silicon host. Full stochastic experiments use seeds 0-4. Systems use repeated native/guest workloads. The shared runner records timestamp, platform, dependency versions, source hashes and thread settings. Run: uv sync --frozen; uv run python -m pytest; uv run python -m optlab run --track all; uv run python -m optlab report. xv6 also requires QEMU and a RISC-V toolchain; see README.

Quick mode writes results/quick and never overwrites full results. Raw data, source references, detailed formulas and the complete Chapter 6 checklist accompany this report. Timings are host observations, not cross-machine guarantees.

Source map: optlab/track_[a-j].py; tests/; results/[A-J].json; docs/COVERAGE.md; docs/numerical.md; docs/quantum.md; docs/systems.md; docs/COMBINATORIAL_DATABASE_SOFTWARE.md.

How to read the results

Read each track in the same order: objective, experimental design, correctness limits, measured comparison and practical interpretation. Objective quality and runtime answer different questions. An exact method may return a better solution while taking longer; a faster reduced test suite may retain line coverage while missing a fault.

Portfolio findings

The strongest systems change is the xv6 fork workload: the median drops from 81 to 8 guest ticks, while CPU and mixed workloads remain unchanged. Compiler passes reduce static instructions and native time on three programs. Database index selection trades storage for lower workload latency. The quantum and parallel experiments retain negative results instead of turning simulation or process overhead into unsupported speedup claims.

Track A - Continuous optimization

Source code | Full measured results

Objective

Minimize regression/ML loss over continuous parameters; compare convex, strongly convex and nonconvex landscapes.

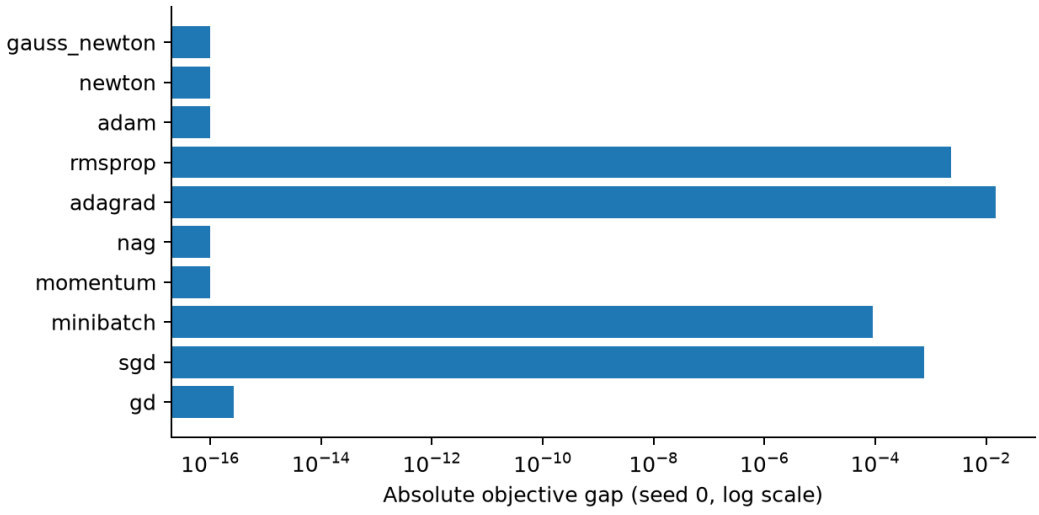
Method and experimental design

Five seeded datasets; identical initial parameters. GD, SGD, mini-batch, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton and Gauss-Newton are implemented. A nonlinear exponential residual distinguishes the two curvature methods. Manual neural-network backpropagation is checked independently of GD/Adam. Fisher, diagonal/low-rank approximations and Hessian-vector products are measured. Validation selects step size; noise and L2 regularization are varied.

Correctness and limits

Least-squares reference and finite-difference derivative checks establish correctness. Equal update counts are not equal sample-gradient budgets; counts are retained. Saddle initialization can stall and aggressive rates diverge. These small tasks do not establish a universal best optimizer.

Method (strongly_convex)	Mean objective gap	Mean seconds
gd	5.395e-17	0.0041123
sgd	0.0013433	0.0031295
minibatch	0.0001374	0.0031612
momentum	-2.3419e-18	0.0044339
nag	-9.541e-19	0.0048065
adagrad	0.047321	0.0048283
rmsprop	0.0030051	0.0051639
adam	-1.648e-18	0.0060959
newton	-1.0408e-18	0.0084204
gauss_newton	-1.0408e-18	0.0083359



Interpretation and practical use

Use the objective gap to compare solution quality and the recorded gradient work to judge computational effort. Curvature can accelerate a small smooth problem but requires additional linear algebra. Mini-batching changes noise as well as cost. For a larger model, compare at matched gradient-work budgets and use validation data to choose learning rates before interpreting held-out performance.

Evidence: results/A.json. Full-run source hash: 7fc7af727aa69ca7. Execution UTC: 2026-10-15T04:22:54.117143+00:00.

Track B - Discrete optimization

Source code | Full measured results

Objective

Maximize knapsack value under integer capacity; demonstrate exact and heuristic discrete optimization and QUBO/Ising conversion.

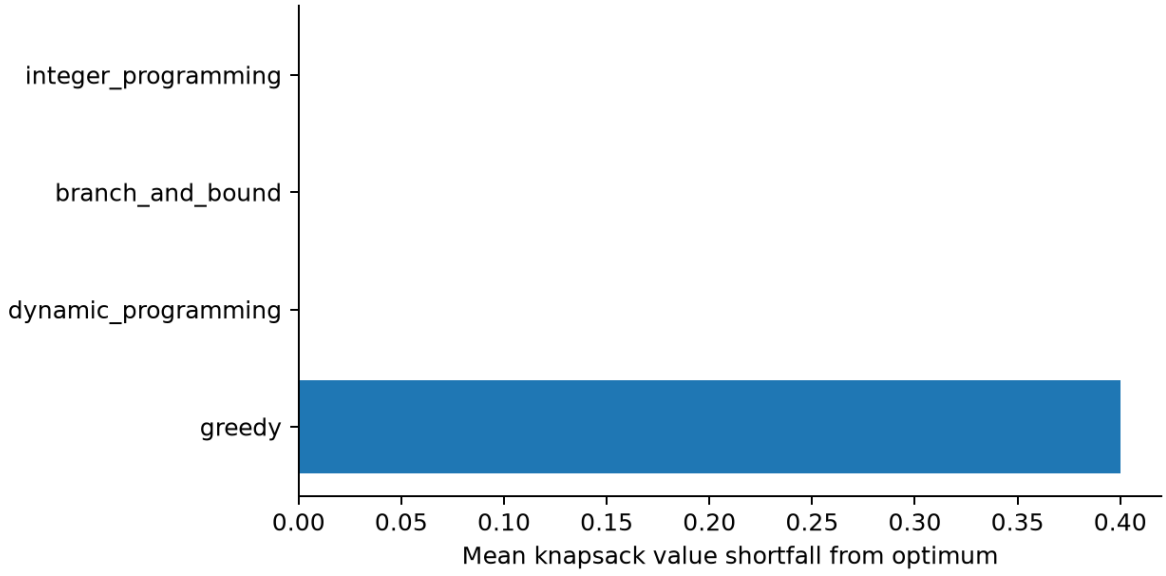
Method and experimental design

Five 24-item instances compare ratio greedy, dynamic programming, branch-and-bound and ILP. Each timing has a warm-up and repeated samples. Exact methods are checked against exhaustive tiny-instance enumeration. Four-cycle Max-Cut is represented as QUBO and Ising energies; all 16 assignments agree. Cardinality-two selection demonstrates a penalty exceeding possible objective gains.

Correctness and limits

Greedy is not exact. DP is pseudo-polynomial; storing reconstruction tuples adds path-copy costs. Branch-and-bound uses exact rational ordering and integer bounds after review exposed float precision errors. ILP uses floating-point coefficients on these small bounded integers.

Method	Mean optimum gap	Mean median seconds
greedy	0.4	2.0142e-05
dynamic_programming	0	8.76e-05
branch_and_bound	0	5.2958e-05
integer_programming	0	0.0031667



Interpretation and practical use

The mean gap measures value sacrificed relative to the exact reference, not a percentage of runtime saved. Greedy is a useful inexpensive feasible baseline; exact DP is attractive when capacity is moderate. Branch-and-bound benefits from tight upper bounds but has exponential worst cases. The shared four-cycle construction makes the classical and quantum objectives directly comparable.

Evidence: results/B.json. Full-run source hash: eeac649ba0fc983e. Execution UTC: 2026-10-15T04:38:12.121379+00:00.

Track C - Quantum optimization

Source code | Full measured results

Objective

Optimize parameterized quantum circuits with classical feedback; compare objective quality, circuit counts and measurement cost.

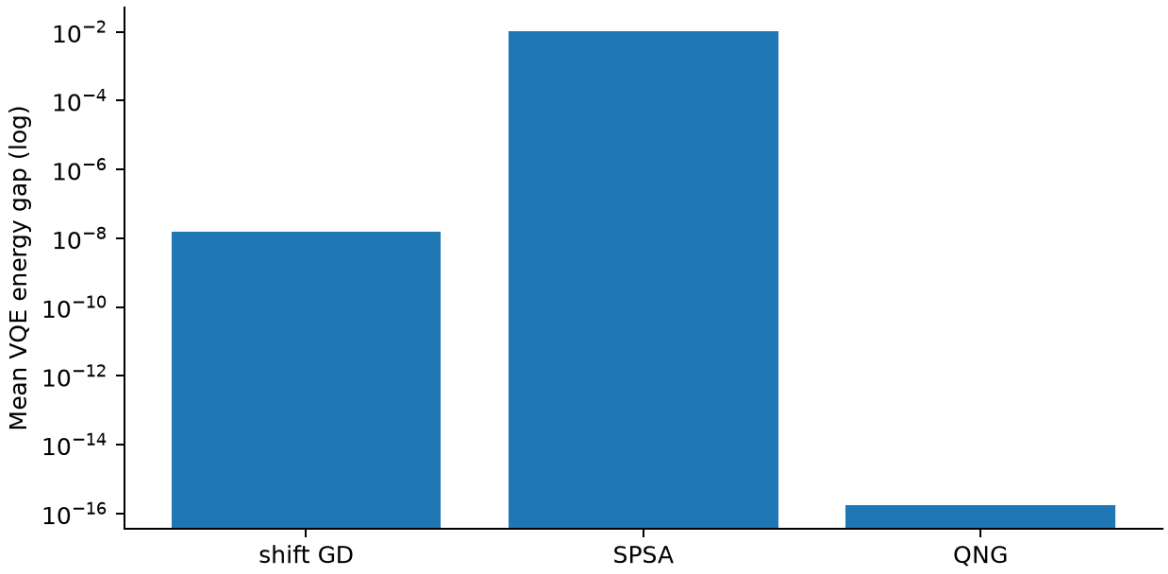
Method and experimental design

Five seeds: p=2 QAOA solves the same four-cycle as B, VQE minimizes Z0+Z1+0.5 X0X1, and a PQC regresses sin(x). Parameter-shift gradient descent, SPSA and regularized quantum natural gradient share initial conditions. A density-matrix depolarizing hardware-noise model and finite-shot sampling study measurement error. Random circuits at 2/4/6/8 qubits probe gradient variance.

Correctness and limits

QAOA is bounded by exact enumeration and VQE by diagonalization. QNG state-derivative overhead is counted separately. Statevectors/density matrices run on a classical CPU, not a QPU. Small size/depth sweeps do not prove asymptotic barren plateaus; clean-angle evaluation under noise is not noisy retraining.

Experiment	Measured value	Meaning
QAOA	4	exact cut 4
VQE	1.3323e-15	max absolute energy error
parameter_shift_gradient	1.5501e-08	mean energy gap
SPSA	0.010345	mean energy gap
quantum_natural_gradient	1.7764e-16	mean energy gap



Interpretation and practical use

A smaller energy gap indicates a better variational solution, but it does not by itself imply a cheaper computation. Parameter-shift requires repeated expectations and QNG additionally estimates circuit geometry. Finite shots introduce sampling variability; depolarization changes the state being measured. A practical comparison must account for those evaluation costs alongside the final objective.

Evidence: results/C.json. Full-run source hash: 357a01c194f8af3d. Execution UTC: 2026-10-15T04:22:55.862945+00:00.

Track D - Submodular, parallel and distributed

Source code | Full measured results

Objective

Maximize submodular coverage; reduce parallel makespan; study communication and conflicting learning objectives.

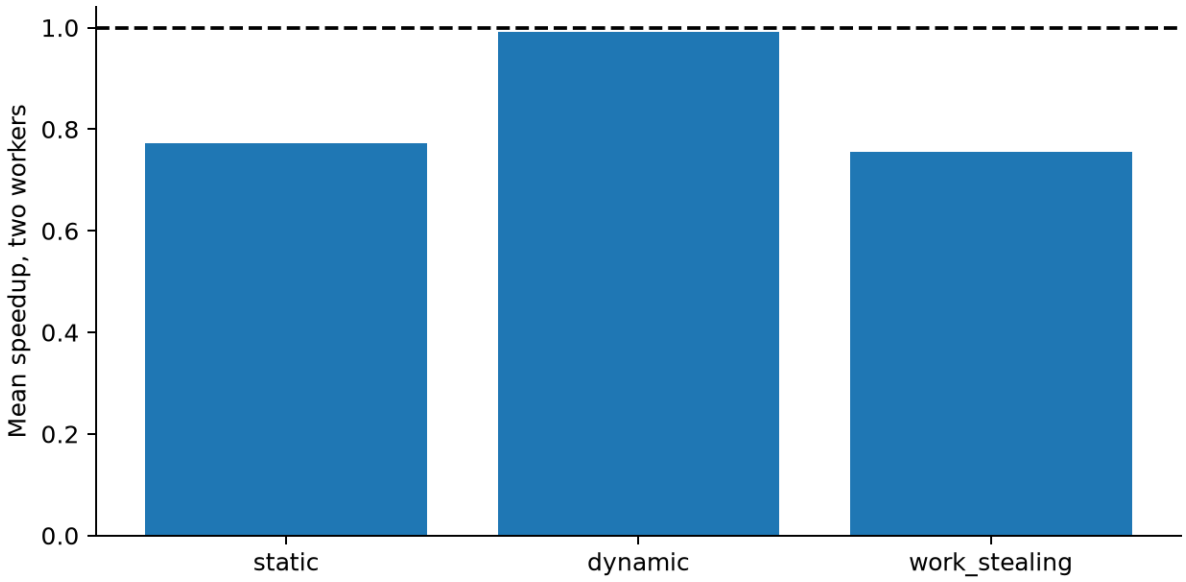
Method and experimental design

Coverage greedy is checked against exhaustive optima and diminishing-return conditions, including a 5/6 counterexample. Real spawned processes execute static, dynamic and work-stealing scheduling, locality experiments and data/model/pipeline decompositions. FedAvg varies local steps and rounds against centralized gradient work. MGDA and GradNorm balance a two-task shared model against equal weights.

Correctness and limits

Parallel outputs match serial references. Timings include spawn, IPC, synchronization and shutdown; small tasks often slow down. FedAvg clients are sequential simulations, not a measured network. Amdahl and Gustafson predictions use stated assumed serial fractions; actual speedup and efficiency are separate measurements.

Scheduler	Mean speedup	Mean efficiency
static	0.77181	0.3859
dynamic	0.99119	0.49559
work_stealing	0.75508	0.37754



Interpretation and practical use

The dashed speedup-one reference separates improvement from slowdown. Process creation and communication can dominate small jobs even when work is balanced correctly. More substantial tasks may amortize this fixed cost, but that is a new experiment. FedAvg and multi-task balancing address different constraints: communication frequency versus disagreement between task gradients.

Evidence: results/D.json. Full-run source hash: 49fb5926f02d7572. Execution UTC: 2026-10-15T04:22:58.958606+00:00.

Track E - Campus shuttle routing

Source code | Full measured results

Objective

Reduce the travel distance of a simulated campus shuttle while serving every stop within capacity six.

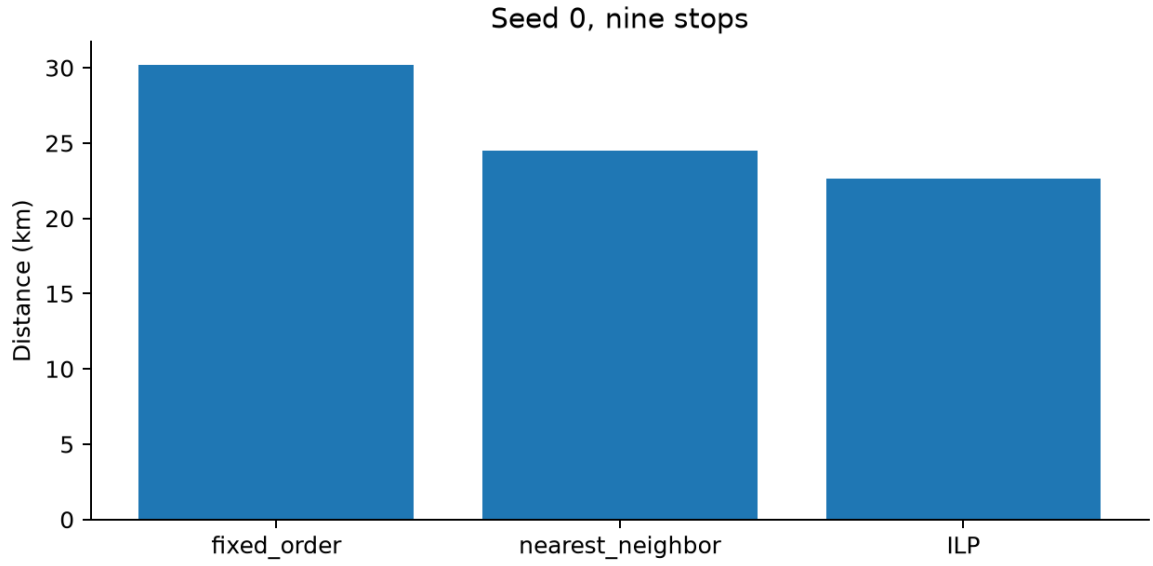
Method and experimental design

Synthetic baseline: index-order visitation, returning to the depot when necessary. Five seeds create 5- and 9-stop datasets in a six-kilometre square with demand 1-3. Compare feasible nearest-neighbor routes with binary-arc ILP using load-based MTZ constraints. Minimize sum(d_ij*x_ij); one arrival/departure per stop, balanced depot flow, q_i <= u_i <= Q and u_i-u_j+Q*x_ij <= Q-q_j.

Correctness and limits

Every route is checked for unique stop coverage and capacity. Four-stop solutions are checked against independent permutation/partition enumeration. Solver statuses, bounds and gaps are retained with a 20-second limit. This is simulated evidence, not an actual campus survey; roads, time windows and fleet limits are excluded.

Stops	Method	Mean distance km
5	fixed_order	19.924
5	nearest_neighbor	16.761
5	ILP	15.752
9	fixed_order	31.281
9	nearest_neighbor	25.39
9	ILP	22.653



Interpretation and practical use

The table averages distance across five seeds at each size; the plot shows one nine-stop instance rather than a second aggregate. Capacity-aware nearest neighbor makes inexpensive local choices. The ILP coordinates all arcs and load constraints together. For deployment, replace straight-line distance with road travel times and add the actual fleet and service-window constraints.

Evidence: results/E.json. Full-run source hash: 12a6e7ed03adcd3c. Execution UTC: 2026-10-15T04:18:28.290445+00:00.

Track F - Compiler optimization

Source code | Full measured results

Objective

Reduce generated code cost without changing unsigned 64-bit program behavior; optimize compiler pass ordering.

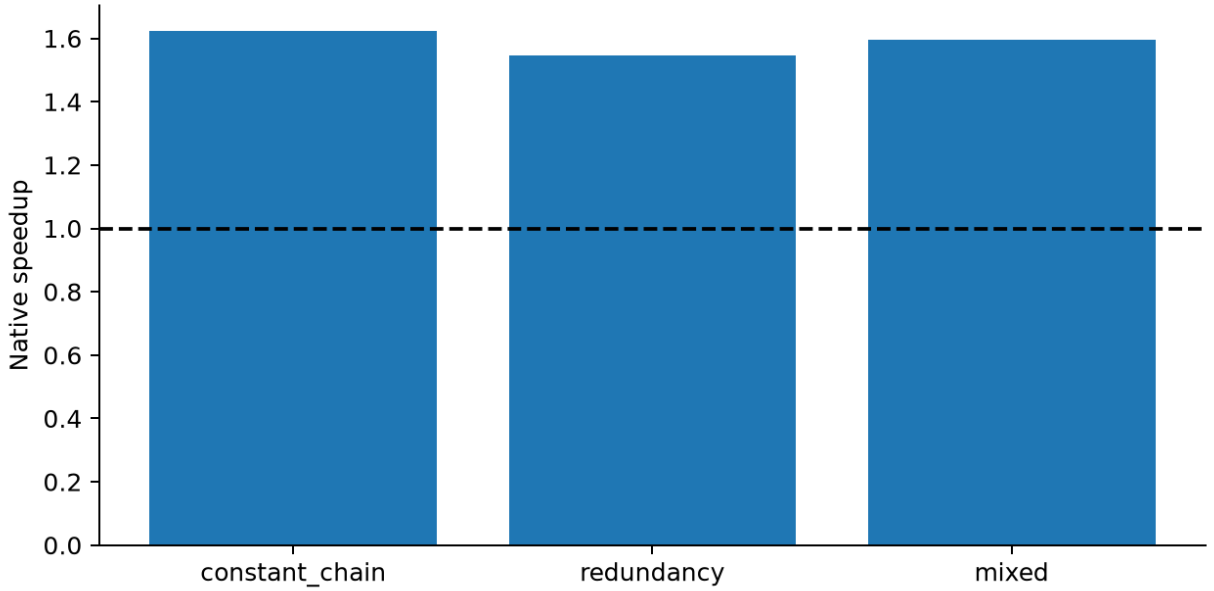
Method and experimental design

A toy-language lexer/parser produces temporary-based IR, interpreted and compiled to C. Four passes: constant folding, propagation, dead-code elimination and common-subexpression elimination. Greedy triple-pass lookahead minimizes weighted IR cost, subject to output equivalence. Native Clang -O0 executables run 100 million inputs, with five CPU-time samples per variant.

Correctness and limits

All 24 pass permutations are checked on representative programs; native checksums match. Frontend, optimization search, backend and end-to-end compilation costs are recorded after review found omitted overhead. Static function instruction counts are not retired hardware counters; executable padding can hide size reductions.

Program	Native speedup	Instructions before / after
constant_chain	1.6228	19 / 9
redundancy	1.5465	25 / 12
mixed	1.5969	35 / 15



Interpretation and practical use

Speedup is baseline runtime divided by optimized runtime; a value above one favors optimization. Instruction reduction and timing support the same conclusion on these programs, but compilation overhead must also be paid. Pass-order search is therefore worthwhile only when the generated program runs enough times to recover the extra compilation work.

Evidence: results/F.json. Full-run source hash: 7a75e3042dd3242d. Execution UTC: 2026-10-15T04:24:05.173995+00:00.

Track G - xv6 operating-system optimization

Source code | Full measured results

Objective

Optimize xv6 memory sharing and scheduling while preserving kernel correctness.

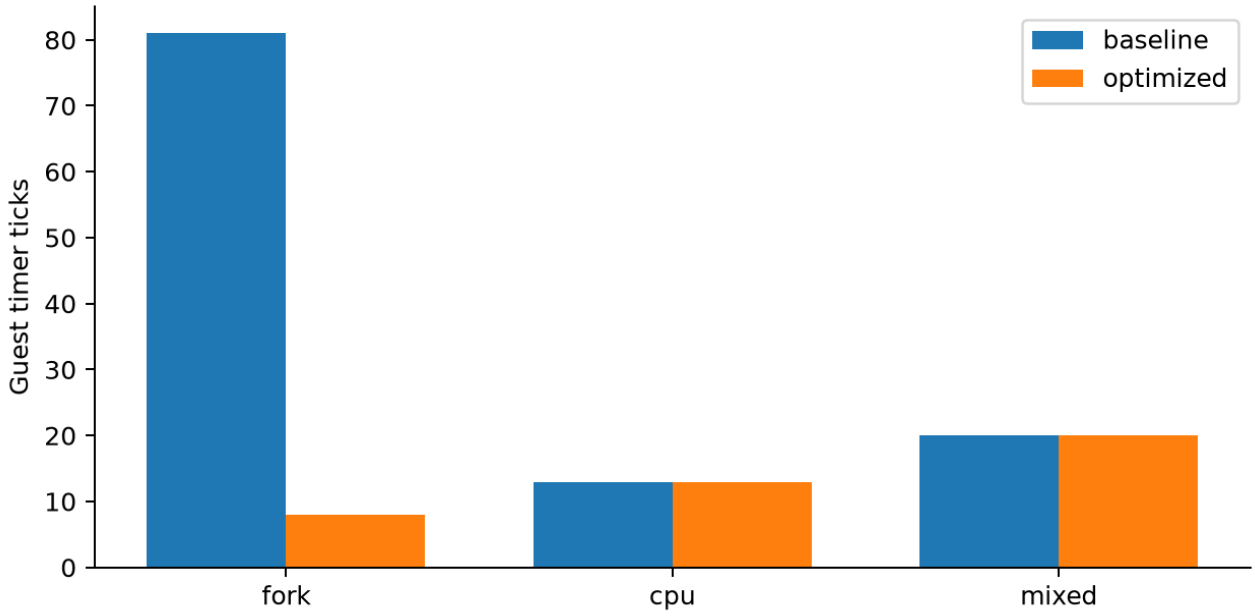
Method and experimental design

Pinned MIT xv6-riscv runs in QEMU. Baseline round-robin/eager fork is compared with equal-ticket lottery scheduling and copy-on-write fork. Page-reference counts, store faults and kernel copyout handle COW; read-only pages remain protected. Three repetitions each of fork-heavy, CPU and mixed CPU/sleep workloads are measured in guest timer ticks.

Correctness and limits

Fresh-disk full upstream usertests pass for baseline one hart, optimized one hart and optimized three harts; raw logs are retained. Two simultaneous kernel changes prevent clean causal attribution. Fork gains plausibly come from avoided copying. No scheduler fairness advantage is claimed. QEMU timing is not physical RISC-V performance.

Workload	Baseline ticks	Optimized ticks
fork	81	8
cpu	13	13
mixed	20	20



Interpretation and practical use

The fork-heavy median improves from 81 to 8 ticks, a 10.125x ratio; the CPU and mixed medians remain 13 and 20 ticks. This concentrates the measured benefit in the workload affected by copying. To isolate scheduler effects, a future ablation would compare each feature separately under controlled ticket allocations and report fairness as well as throughput.

Evidence: results/G.json. Full-run source hash: 0d547d6fe3619034. Execution UTC: 2026-10-15T04:31:30.716611+00:00.

Track H - Database index selection

Source code | Full measured results

Objective

Choose database indexes to reduce a real SQLite query workload under a storage budget.

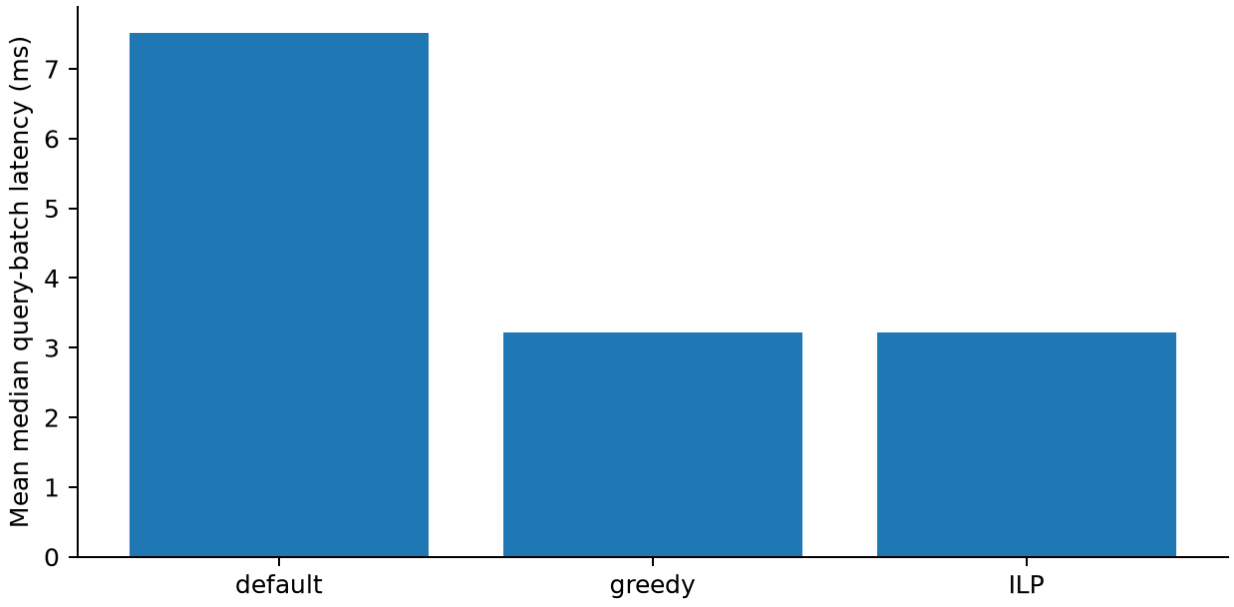
Method and experimental design

Five synthetic 50,000-row sales databases and five fixed SQL queries compare default no-secondary-index planning with greedy and ILP index selection. Candidate sizes are measured occupied pages. The binary objective sums isolated latency gains subject to 40% of total candidate storage. Chosen combinations are then actually benchmarked; EXPLAIN QUERY PLAN and exact result hashes are retained.

Correctness and limits

Integer currency avoids floating-point aggregation-order changes. Median warm-cache latency, raw samples, sequential throughput and storage are measured. Additive gain estimates ignore index interactions, so ILP optimality applies to the selection model, not necessarily measured runtime. Normalized objective units avoid solver-tolerance errors.

Index policy	Mean latency ms	Mean storage bytes
default_no_secondary_indexes	7.5123	917504
greedy	3.2145	1994752
ILP	3.2164	1994752



Interpretation and practical use

The measured latency comes from actually running the selected index set, not simply summing estimated gains. Storage is a real constraint because each candidate consumes database pages. An additive model is convenient for greedy and ILP comparison but cannot capture every planner interaction. A production study would include write overhead, cold caches and workload drift.

Evidence: results/H.json. Full-run source hash: 4f13cc3820e0abfb. Execution UTC: 2026-10-15T04:23:01.010446+00:00.

Track I - Quantum algorithms and QML

Source code | Full measured results

Objective

Measure quantum algorithm behavior and QML tradeoffs relative to classical methods.

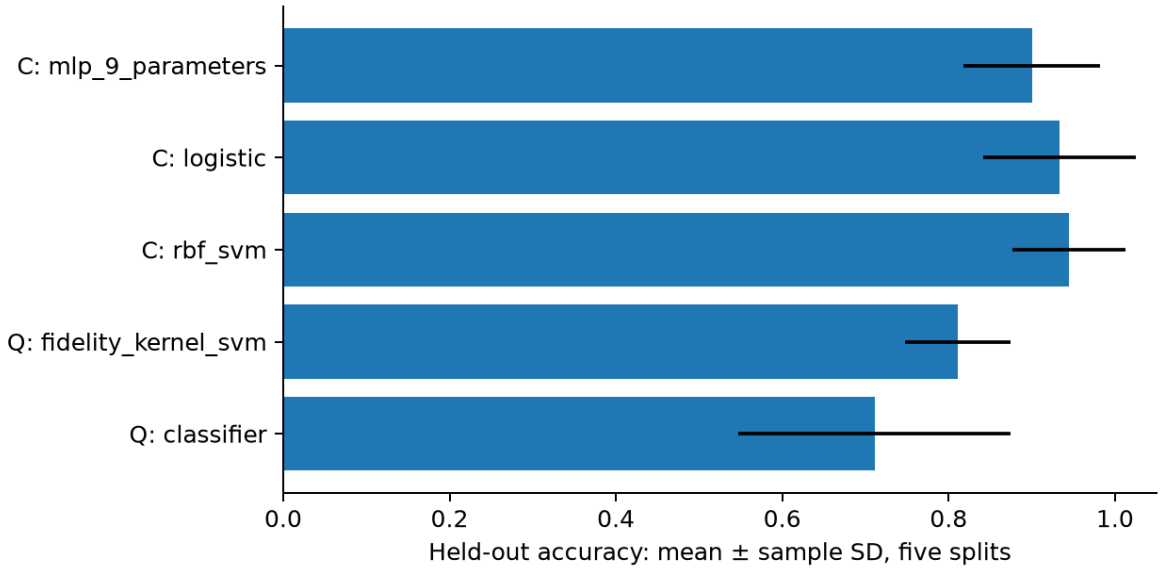
Method and experimental design

Direct circuits demonstrate Deutsch-Jozsa, Bernstein-Vazirani, Grover, QFT, QPE and Shor order finding for 15. Suzuki-Trotter Monte Carlo models annealing and is compared with classical annealing. A nine-parameter variational classifier and fidelity-kernel SVM share train/test rows with classical models on 72 two-moons samples. Scalers fit training data only.

Correctness and limits

Grover includes actual linear-search comparisons, logical gate/query counts and separately labeled simulator timings. QML reports accuracy, training time, parameters and circuit evaluations; cached feature-state work differs from hardware overlap circuits. Classical models outperform these quantum classifiers on average. Dense Shor multiplication and simulated annealing are small demonstrations, not scalable hardware implementations.

Model	Mean test accuracy	Sample SD
variational_quantum_classifier	0.71111	0.16387
quantum_fidelity_kernel_svm	0.81111	0.063343
classical_rbf_svm	0.94444	0.068041
classical_logistic	0.93333	0.091287
classical_mlp_9_parameters	0.9	0.082402



Interpretation and practical use

Accuracy is averaged over five data splits and the error bars show sample standard deviation. Shared rows and train-only scaling prevent a favorable split or preprocessing leakage from driving the comparison. The simulator has no demonstrated wall-clock advantage here. Grover query savings concern oracle use, while dense state simulation introduces a separate classical cost.

Evidence: results/I.json. Full-run source hash: 843409ac3bdeacc0. Execution UTC: 2026-10-15T04:21:58.919044+00:00.

Track J - Software-engineering optimization

Source code | Full measured results

Objective

Minimize test execution cost while preserving all Python JSON lines covered by a candidate suite.

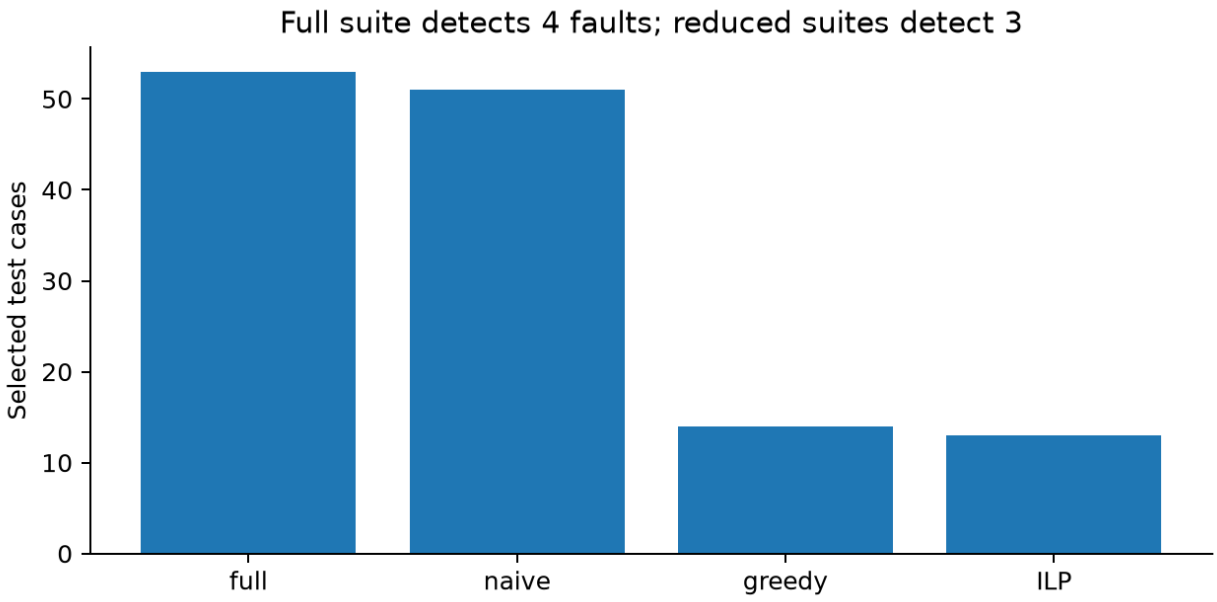
Method and experimental design

The target is public CPython json (1,317 Python source lines), not the small toy compiler. Fifty-three cases cover parsing, encoding, Unicode, invalid syntax and options. cProfile establishes baseline costs. Weighted set cover compares full suite, naive prefix, greedy and ILP. Minimize sum(t_i*x_i) subject to each observed line retaining at least one selected test.

Correctness and limits

All selected suites preserve observed line coverage, but reduced suites miss one of four deliberate API mutations detected by the full suite. This proves coverage retention alone does not guarantee behavior retention. C accelerator lines are not traced. Microsecond timings are noisy and selection overhead may require many CI runs to amortize.

Suite	Tests / speedup	Detected faults (of 4)
full	53 / 0.999x	4
naive	51 / 1.137x	3
greedy	14 / 2.953x	3
ILP	13 / 3.010x	3



Interpretation and practical use

The full suite contains 53 cases; greedy retains 14 and ILP retains 13. Both retain observed line coverage while detecting only three of four injected faults, compared with four for the full suite. Use reduction as a cost-quality decision rather than a correctness guarantee, and consider periodically running the full suite to catch behavior absent from the coverage model.

Evidence: results/J.json. Full-run source hash: 08335dd453465843. Execution UTC: 2026-10-15T04:18:30.418143+00:00.

AI comparison and development failures

Original Codex and Gemini outputs passed 100 independently generated small-integer instances, with exhaustive optimum checks for both exact methods. Six Gemini embedded tests pass. Both original branch-and-bound implementations fail on permitted large integers because floating-point upper bounds can prune a better solution. Corrected Codex uses exact rational ordering and integer bounds; accepted NumPy integers are normalized to Python integers to prevent overflow.

The comparison is not a controlled ranking of foundation models: Codex had assignment context and its original module imports NumPy/SciPy; Gemini received a standard-library-only prompt. Corrected Codex has the benefit of review. The original response claims approximately 2.5 seconds; this is unverified response latency. The Antigravity workflow statement is supplied by the student.

Original implementation	Small exact checks	Adversarial issue
Codex original	200/200	float bound precision
Gemini original	200/200	float bound precision

Measured timing samples and preserved outputs are in prompts/second-tool/COMPARISON.md and results/AI_COMPARISON.json. Greedy ties may legitimately yield different answers. Tuple-based DP reconstruction costs more than the supplied O(C)-space claim; original analysis also overstated how hard some branch-and-bound benchmarks were.

Other genuine AI errors: database ILP coefficients were too small for solver tolerance; compiler timing omitted optimization search; Grover comparison omitted classical execution. Each was independently reproduced or inspected, corrected and regression-tested. docs/AI_FAILURE_LOG.md links the evidence. Deliberate mutation faults are kept separate from real AI mistakes.

What the corrections changed

The branch-and-bound fix changes numerical reasoning: rational ratio ordering and integer upper bounds prevent a feasible better branch from being discarded by rounding. Database gains are normalized before ILP selection so solver tolerances do not erase meaningful differences. Compiler measurements now include frontend, pass-search and backend costs. Grover reporting separates oracle complexity from the time needed to simulate quantum states.

Verification protocol

The comparison uses 100 seeded instances with one to twelve items and an exhaustive subset reference. It checks 200 exact objective values, 300 feasibility conditions and 33 invalid-input cases per implementation. Separate adversarial inputs test precision beyond the small-instance sample. Runtime is evaluated on the same 50 twenty-item cases, after warm-up, in seven batches; it is distinct from time spent generating code.

Disclosure and responsibility

Codex model label from local execution records: gpt-6-astra. Work: planning, code, tests, debugging, review, report; tracks A-J. Supplied Gemini label: Gemini 3.8 Flash (High); work: Track B comparison and draft documentation, corrected after review. Prompt records, visible conversation excerpts, source changes and AI-marked integration commits are retained. No secret credentials are included.

Answers to the assignment's final questions

1. Process and inefficiency: simulated campus shuttle routing; index-order routes provide an explicit measured baseline. The recorded dataset makes the choice reproducible.

2. Formulation: minimize total Euclidean arc distance under unique stop visitation, positive indivisible demand, depot flow and vehicle-capacity constraints. See E for equations.

3. Methods: continuous gradients/curvature for smooth losses; discrete exact/heuristic methods for combinatorial choices; submodular greedy only where diminishing returns holds; hybrid quantum methods on suitable small circuits. Coverage maps each required method to code/results.

4. Improvement: consult each measured table. Routing/compilation/index selection/COW yield gains on these workloads; some quantum and parallel cases lose. No single algorithm is universally best.

5. Tradeoffs: exact ILP search costs time; greedy sacrifices quality; process startup can dominate; QML simulation/measurement costs matter; coverage reduction may sacrifice behavioral fault detection.

6. Evidence: analytical/exhaustive references, derivative checks, equivalent native outputs, exact SQL results, full xv6 usertests, fixed seeds, raw samples and locked dependencies.

7. Adaptation: larger instances may need heuristic/local-search methods, sparse curvature, real road constraints, held-out workloads, distributed hardware and noise-aware models. Change the formulation when constraints change.

8. Systems concepts: compiler IR transformations and pass ordering; xv6 page tables/reference counting/fault handling and scheduling; database access paths/storage budgets. Correctness is verified before performance claims.

9. Quantum/parallel advantages and limits: Grover reduces oracle queries but classical simulation is slower; QML accuracy is lower here; parallel results expose startup/IPC overhead. Theoretical, simulated and hardware measurements are explicitly distinguished.

10. AI tools: Codex and supplied Gemini. Review exposed solver scaling, omitted compiler/search timing, incorrect large-integer pruning and unsupported documentation. Original outputs and fixes are preserved.

Primary references

MIT xv6 source and COW lab: github.com/mit-pdos/xv6-riscv; pdos.csail.mit.edu/6.1810/2024/labs/cow.html

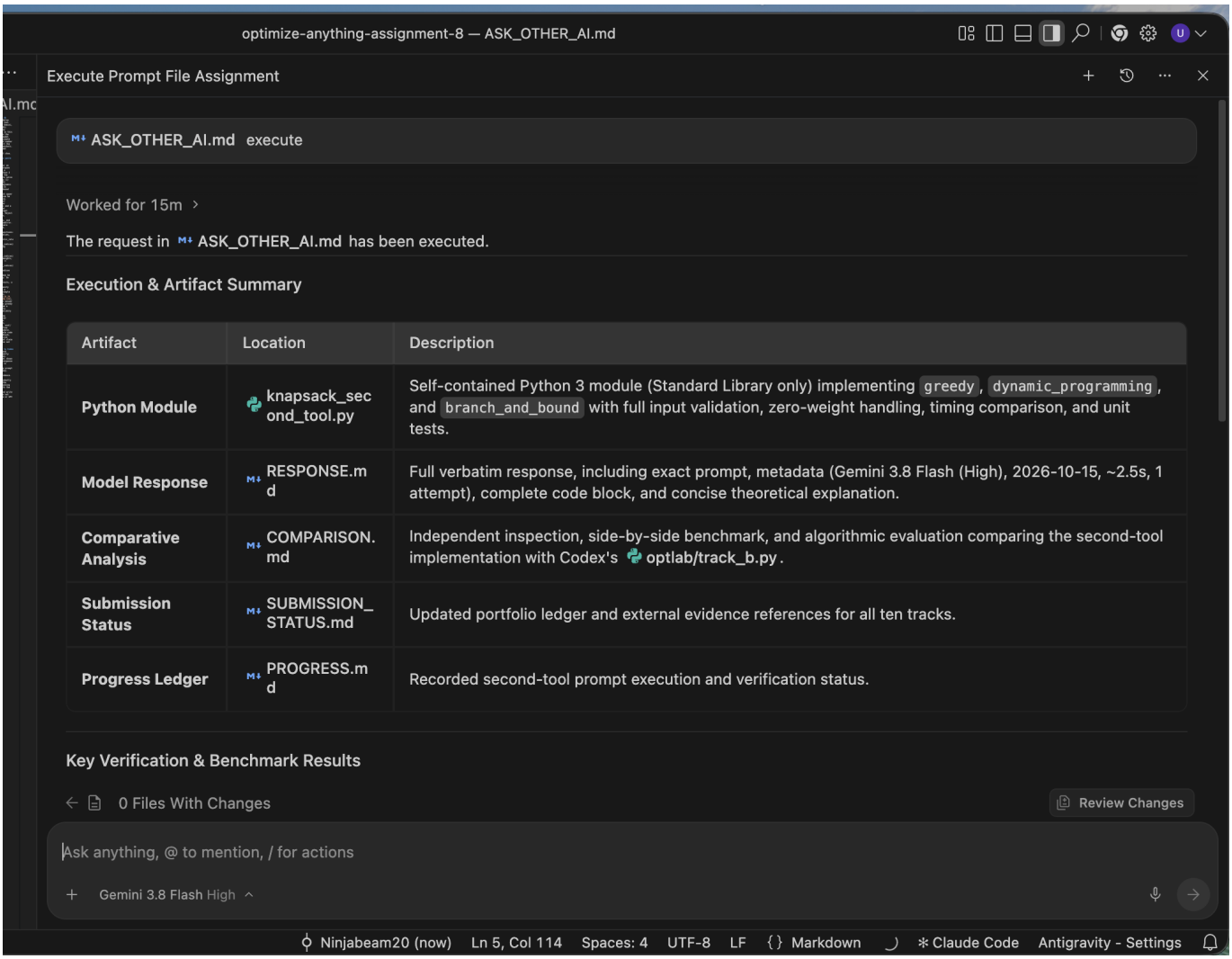
SciPy MILP documentation: docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html; SQLite query plans: sqlite.org/eqp.html

Adam: arxiv.org/abs/1412.6980; FedAvg: proceedings.mlr.press/v54/mcmahan17a.html; MGDA: arxiv.org/abs/1810.04650; GradNorm: proceedings.mlr.press/v80/chen18a.html

Quantum reference links and implementation caveats: docs/quantum.md. CPython JSON source/license: github.com/python/cpython/tree/v3.12.13/Lib/json; docs.python.org/3/license.html

Gemini evidence 1 - Prompt and response

Original user-supplied screenshot: prompt-file execution, selected Gemini 3.8 Flash High label and generated-artifact summary. The UI displays Worked for 15m; the original response claims approximately 2.5 seconds. These are different reported durations, not independently measured code-generation latency.



Evidence interpretation

The model selector and prompt-file execution connect the supplied implementation to the second-tool workflow. The screenshot records the interface and response; correctness is established separately by executing the preserved code against an exhaustive oracle. Reported timing claims are not treated as independently measured model latency.

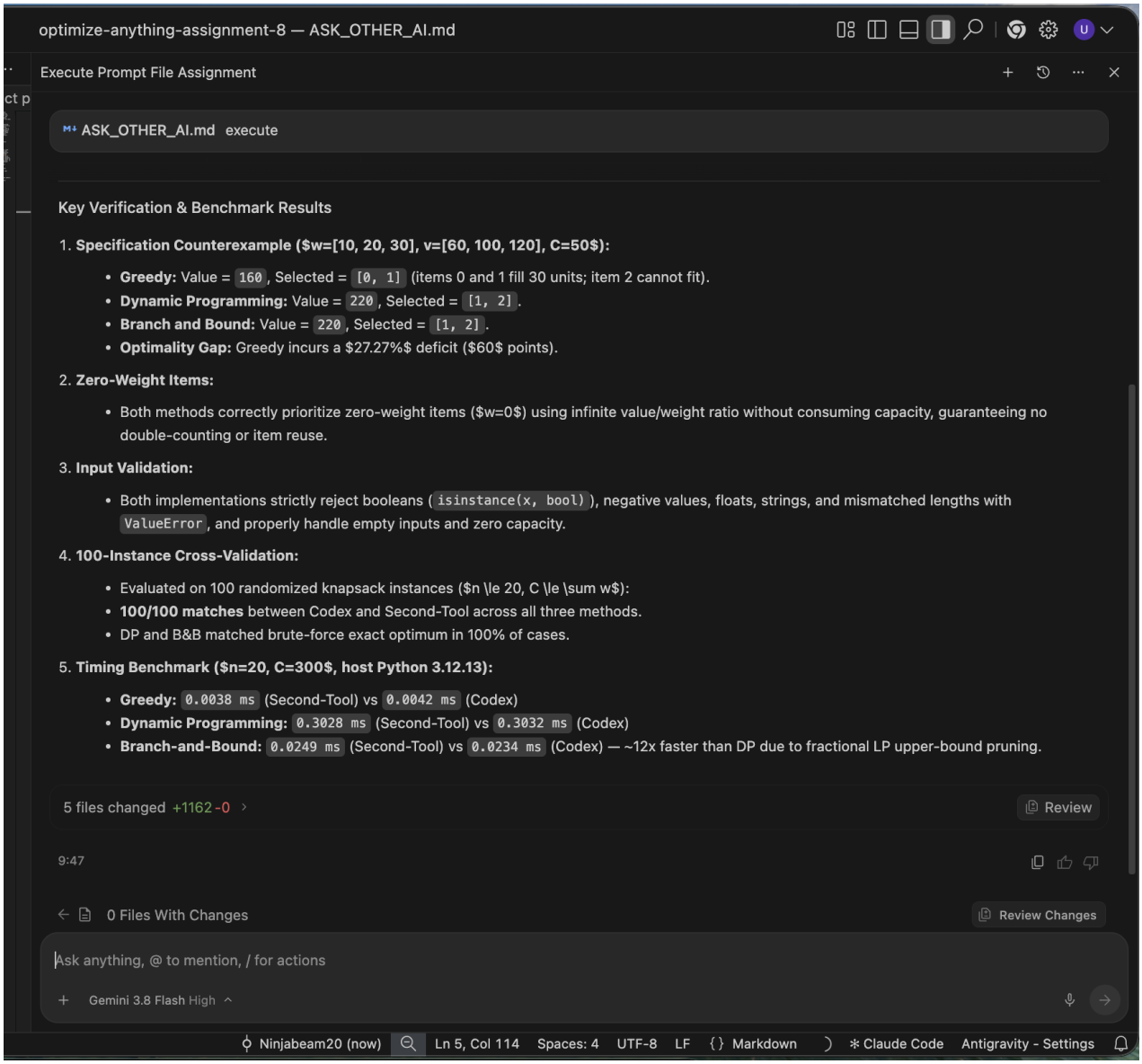
Connection to the submitted files

The request names ASK_OTHER_AI.md, and the generated artifacts correspond to the preserved response and knapsack module in prompts/second-tool/. The independent comparison and raw AI_COMPARISON.json results document what was checked after generation, including large-integer counterexamples that a screenshot alone cannot reveal. The original image and code remain available for inspection alongside the corrected project implementation.

Original screenshot is retained unchanged; its SHA-256 is recorded in prompts/second-tool/SCREENSHOTS_SHA256.json.

Gemini evidence 2 - Benchmark claims

Original user-supplied screenshot: Gemini reports its knapsack example, validation, 100-instance cross-check and timing claims. This is evidence of the response, not independent verification. Our exhaustive checks and large-integer counterexamples are in results/AI_COMPARISON.json and prompts/second-tool/COMPARISON.md.



Evidence interpretation

The model selector and prompt-file execution connect the supplied implementation to the second-tool workflow. The screenshot records the interface and response; correctness is established separately by executing the preserved code against an exhaustive oracle. Reported timing claims are not treated as independently measured model latency.

Connection to the submitted files

The request names ASK_OTHER_AI.md, and the generated artifacts correspond to the preserved response and knapsack module in prompts/second-tool/. The independent comparison and raw AI_COMPARISON.json results document what was checked after generation, including large-integer counterexamples that a screenshot alone cannot reveal. The original image and code remain available for inspection alongside the corrected project implementation.

Original screenshot is retained unchanged; its SHA-256 is recorded in prompts/second-tool/SCREENSHOTS_SHA256.json.